TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

A.KESHAVA REDDY (VTU26535)

10211CS224
Full Stack Application Development
Slot : E1-B4

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report ”TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT” by A.KESHAVA REDDY (VTU26535) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty
Dr. Hafizur Rahman
Assistant Professor
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr.Sagunthala R&D
Institute of Science and Technology
May, 2026

Signature of Course Co-ordinator
Dr.Sumithra. M
Assistant Professor(Senior Grade)
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr.Sagunthala R&D
Institute of Science and Technology
May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

A.KESHAVA REDDY

Date:06/05/2026

ABSTRACT

This project develops a React JS–based web application for booking tickets to internal department events such as seminars or technical fests.It provides a simple platform for students and faculty to view event details and book tickets online.The application displays information including event name, date, time, venue, ticket price, and availability.Users can enter their personal details and select the number of tickets required.The system validates all inputs, ensuring mandatory fields, valid email format, and positive ticket count.It prevents users from booking more tickets than the available number.The application updates ticket availability dynamically after each booking.A confirmation message along with booking summary is displayed upon successful booking.The system is built using React concepts like functional components, JSX, useState, and event handling.

Keywords: React JS, Ticket Booking System, Event Management, Functional Components, JSX, useState Hook, Form Validation, State Management, User Interface, Web Application.

LIST OF FIGURES

1.1 Framework 3

3.1 Structure of the Project 6

3.2 Structure of the Project 7

3.3 Backend Implementation 8

4.1 Project Architecture 12

4.2 DataflowDiagram 13

5.1 Login page 16

5.2 Dashboard page 17

5.3 Registration Page 17

5.4 Admin Dashboard 18

5.5 Payment page 19

LIST OF TABLES

5.1 Comparison of Ticket Booking Applications 16

7.1 Mapping of Online Ticket Booking System to Complex Engineering Problem (CEP) 22

7.2 Mapping of Online Ticket Booking System to Complex Engineering Activities (CEA) 23

7.3 SDG Mapping for the Online Ticket Booking System 23

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
DB	Data Base
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
JSX	JavaScript XML
MVC	Model View Controller
QR	Quick Response
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
XML	eXtensible Markup Language

TABLE OF CONTENTS

Page.No

ABSTRACT **iii**

LIST OF FIGURES **iv**

LIST OF TABLES **v**

LIST OF ACRONYMS AND ABBREVIATIONS **vi**

1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Aim of the Project 1

1.3 Scope of the Project 2

1.4 Framework 2

2 SURVEY OF SIMILAR APPLICATION IN MARKET **4**

3 FRAMEWORK DESCRIPTION **5**

3.1 Frontend Implementation 5

3.1.1 Environment Setup 5

3.1.2 Structure of the Project 5

3.1.3 Execution Procedure 6

3.2 Backend Implementation 7

3.2.1 Environment Setup 7
3.2.2 Structure of the Project 7
3.2.3 Execution Procedure 8
3.3 Database Implementation 9
3.3.1 Database Configuration 9
3.3.2 Schema Structure 10
3.3.3 Tables created 10

4 METHODOLOGIES 11
4.1 Project Architecture 11
4.2 DataFlow Diagram 12
4.3 Module 1:Event Details Module 13
4.4 Module 2:Ticket Booking Module 13
4.5 Module 3:Booking Confirmation Module 14

5 RESULTS AND DISCUSSIONS 15

6 CONCLUSION AND FUTURE ENHANCEMENTS 20
6.1 Conclusion 20
6.2 Future Enhancements 21

7 ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG 22
7.1 Mapping of the Project to Complex Engineering Problem (CEP) . . 22
7.2 Mapping of the Project to Complex Engineering Activities (CEA) . 23
7.3 SDG Mapping 23

REFERENCES 24

Chapter 1

INTRODUCTION

1.1 Introduction

This project focuses on developing a React JS–based web application for ticket booking of internal department events such as technical fests or seminars. The application aims to provide a convenient and efficient platform for students and faculty to view event details and book tickets online. It displays essential information like event name, date, time, venue, ticket price, and available tickets, while allowing users to enter their details and reserve tickets easily[1]. The system includes proper validation to ensure accurate inputs and prevent overbooking, along with real-time updates of ticket availability and booking confirmation. By using modern React concepts such as functional components, JSX, useState hook, and event handling, the application offers a responsive, user-friendly interface and demonstrates effective web development practices.

1.2 Aim of the Project

The aim of this project is to design and develop a React JS–based web application that enables students and faculty to easily view event details and book tickets for internal department events online. It focuses on providing a user-friendly inter-

face, ensuring accurate input validation, preventing overbooking, and dynamically updating ticket availability, while demonstrating the effective use of modern React concepts such as functional components, JSX, useState, and event handling.

1.3 Scope of the Project

The scope of this project is to develop a React JS–based web application that facilitates online ticket booking for internal department events such as seminars and technical fests[2]. It includes displaying event details, allowing users to book tickets, validating user inputs, preventing overbooking, and updating ticket availability in real time. The application focuses on a single-event booking system with a simple and user-friendly interface, making it suitable for academic use and small-scale event management. The scope can be further extended to support multiple events, user authentication, payment integration, and database connectivity for real-world deployment.

1.4 Framework

The project is developed using React (JavaScript library), a widely used open-source JavaScript library maintained by Meta Platforms for building dynamic and interactive user interfaces[1]. React is especially suitable for single-page applications (SPA), where fast rendering and smooth user experience are essential.React uses JSX (JavaScript XML), which allows developers to write HTML-like syntax directly within JavaScript[7]. This makes the code more readable and easier to design UI elements. The Virtual DOM is another important concept, which improves performance by updating only the changed parts of the UI instead of reloading the entire page.State management in this project is handled using the useState Hook, which allows components to store and update dynamic data such as user inputs and

available ticket count. When the state changes, React automatically re-renders the component, ensuring that the UI always reflects the latest data.React also supports efficient event handling, enabling the application to respond to user actions like form submission, input changes, and button clicks. Conditional rendering is used to display booking confirmations, error messages, or different UI elements based on user input and system state.Additionally, React promotes a clean project structure with separate files for components, styles, and logic, making the application scalable and easy to maintain. Its fast rendering, strong community support, and flexibility make React an ideal framework for developing modern web applications like this ticket booking system[8].

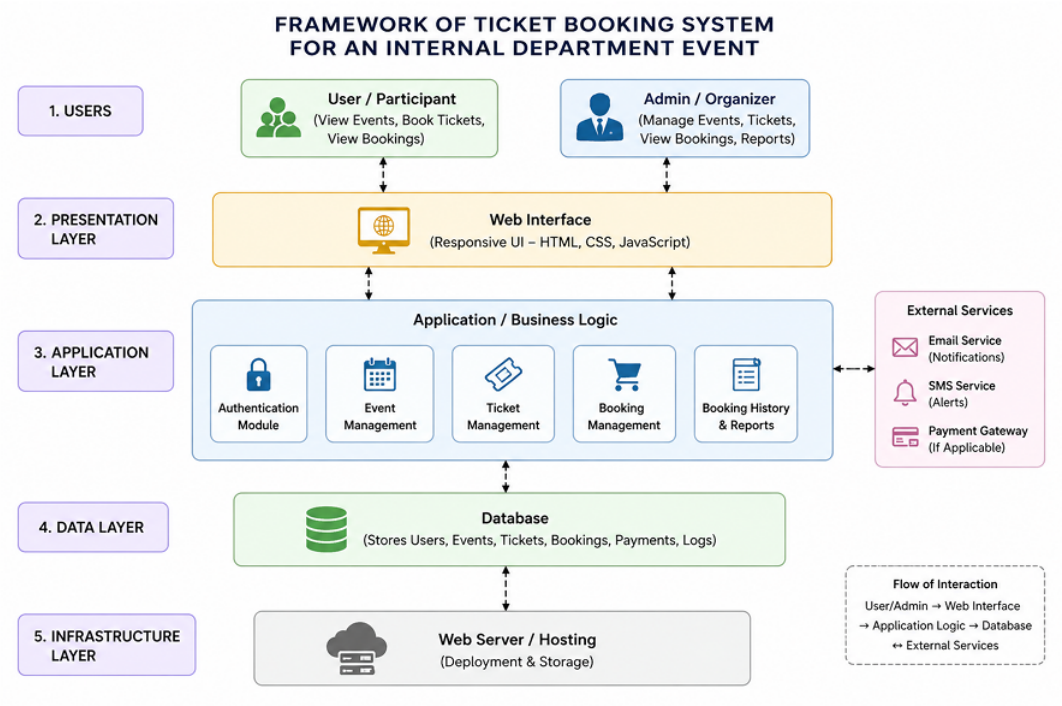

Figure 1.1: Framework

[Figure 1.1] This figure illustrates the overall framework of the ticket booking system. It shows different layers including users, presentation layer, application logic, database, and infrastructure. The diagram explains how users interact with the system through a web interface and how data flows across layers. It also highlights integration with external services like payment and notifications.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Content Several ticket booking applications already exist in the market, providing useful insights for developing this project. One of the most popular platforms is BookMyShow, widely used in India for booking movie, concert, and event tickets with a simple and user-friendly interface[2]. It supports event discovery, seat selection, and secure booking, serving millions of users annually . Another major platform is Eventbrite, which offers event creation, online ticket sales, marketing tools, and analytics to help organizers manage events efficiently .In addition, platforms like Townscript and Paytm Insider focus on local events, workshops, and community activities, making them popular among students and small organizations . These systems commonly provide features such as online booking, real-time ticket availability, secure payments, and user notifications[4]. Many modern applications also include advanced features like analytics dashboards, promotional tools, and mobile-based ticket access.From this survey, it is observed that most existing applications emphasize ease of use, real-time updates, and efficient event management. The proposed project adopts similar core functionalities but focuses on a simplified version tailored for internal department events, ensuring ease of implementation and usability in an academic environment.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the application is developed using React (JavaScript library). To set up the development environment, Node.js and npm are installed on the system[6]. A new React project is created using tools like Create React App or Vite. Required dependencies are installed, and a code editor such as Visual Studio Code is used for development. The project is then initialized and run locally using commands like npm start or npm run dev[6].

3.1.2 Structure of the Project

[Figure 3.1] The project follows a modular and component-based structure. The main folder contains public and src directories. The public folder includes the main HTML file, while the src folder contains all React components, styles, and logic.

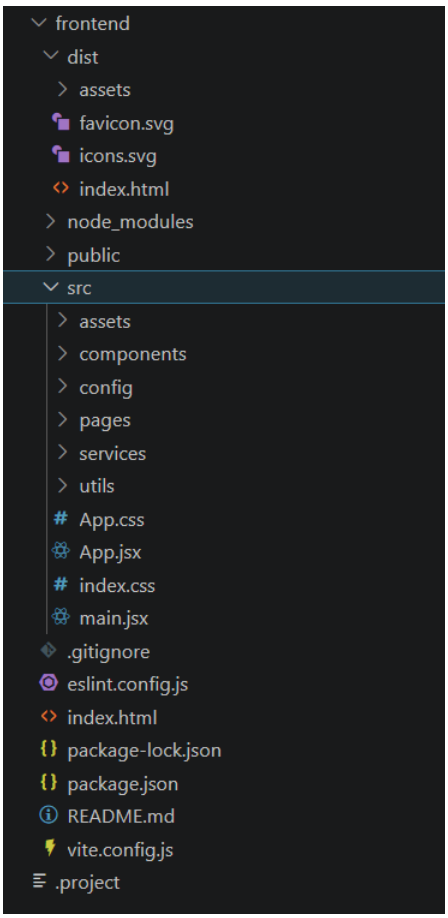

Figure 3.1: Structure of the Project

3.1.3 Execution Procedure

After completing the development, the application is executed by running the project in a local development server. The user opens the browser and accesses the application through a local host URL (e.g., http://localhost:3000). The event details are displayed on the homepage, and users can interact with the booking form. When the form is submitted, validations are performed, and if successful, the booking is processed with updated ticket availability. The output is dynamically rendered on the screen, showing confirmation messages and booking summaries.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend for this project can be developed using Node.js with Express.js to handle server-side logic and API requests. The environment is set up by installing Node.js and the package manager npm[3]. A new project is initialized using npm init, and required dependencies such as Express, body-parser, and CORS are installed. A development tool like Visual Studio Code is used for coding, and tools like Nodemon can be used to automatically restart the server during development.

3.2.2 Structure of the Project

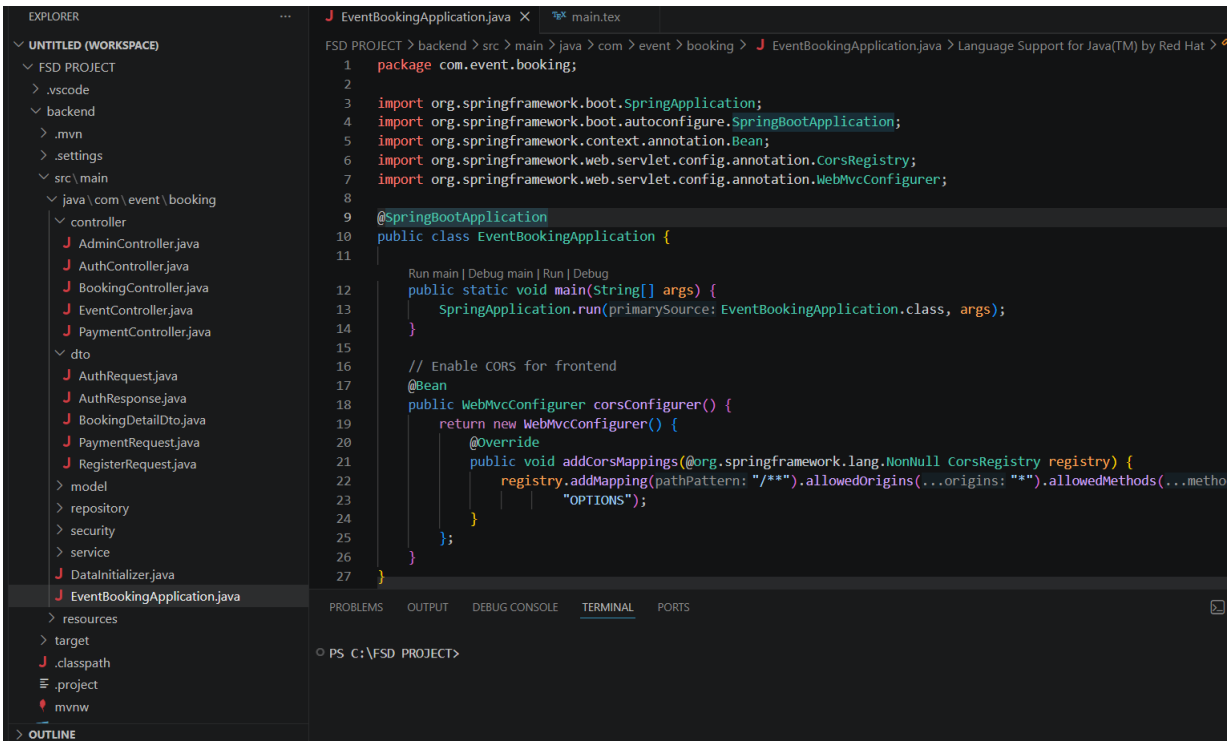

Figure 3.2: Structure of the Project

[Figure 3.2] This figure shows the backend project structure of a Spring Boot application in Visual Studio Code. It includes organized folders such as controllers, DTOs, models, repositories, services, and security for modular development.

3.2.3 Execution Procedure

The backend server is executed by running commands like node server.js or npm start in the terminal. Once the server starts, it listens on a specific port (e.g., http://localhost:500 The frontend React application communicates with the backend through API calls to fetch event details and submit booking data. When a user books tickets, the backend validates the request, updates ticket availability in the database, and sends a response back to the frontend. The system ensures proper error handling and returns appropriate messages for successful or failed operations.

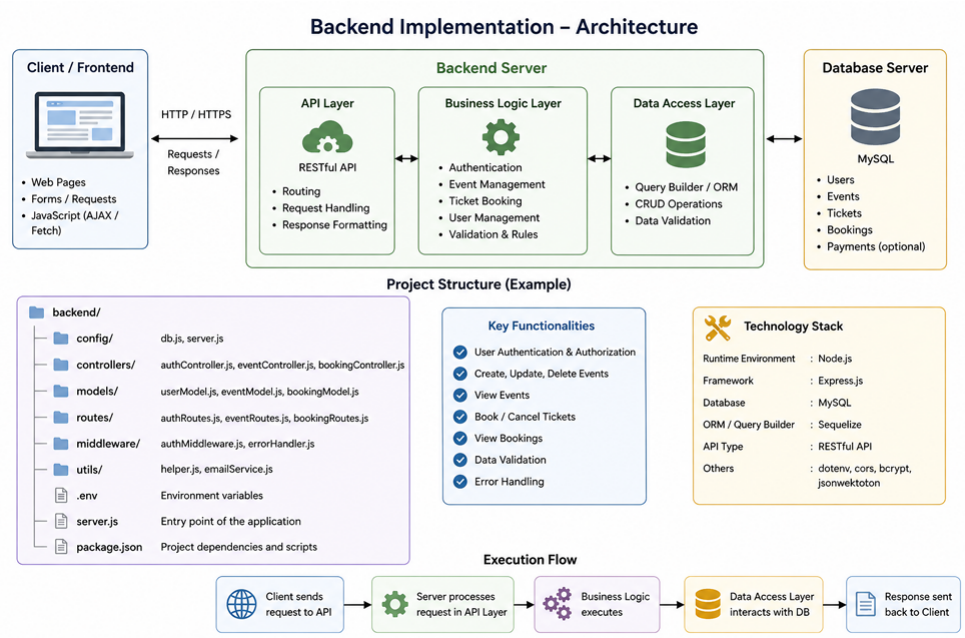

Figure 3.3: Backend Implementation

[Figure 3.3]This figure represents the architecture of the backend system. It shows components such as API layer, business logic, and database interaction. The diagram explains how requests from the frontend are processed and validated before accessing the database. It also illustrates the flow of data between client, server, and database[5].

3.3 Database Implementation

3.3.1 Database Configuration

The database for this project is implemented using MongoDB, a widely used NoSQL database that stores data in a flexible, JSON-like format called BSON (Binary JSON). This structure makes it highly suitable for modern web applications built using technologies like React and Node.js, as it allows seamless data exchange between the frontend and backend.

To integrate MongoDB with the backend server, the project uses Mongoose, which is an Object Data Modeling (ODM) library for Node.js. Mongoose simplifies interactions with the database by providing schema-based solutions, allowing developers to define the structure of data, enforce validation rules, and manage relationships between different collections[7].

The database connection is established in the backend by using a MongoDB connection string (URI). This string typically includes the database host, port number, database name, and authentication credentials if required. The connection is initialized using Mongoose methods, ensuring that the backend server successfully communicates with the database before handling any client requests.

In addition, middleware is used within the backend framework (such as Express.js) to manage HTTP requests and responses efficiently. Middleware functions handle tasks such as parsing incoming JSON data, managing authentication, logging requests, and handling errors. This layered approach improves the overall maintainability and scalability of the application.

The database schema is designed to store essential information such as user details, event data, and booking records. Collections are created for different entities like users, events, and reservations, ensuring proper organization of data. Relationships between these collections are managed using references or embedded docu-

ments, depending on the use case.

Proper indexing techniques are also applied to improve query performance, especially when dealing with large datasets. Security measures such as environment variables are used to store sensitive information like database credentials, preventing unauthorized access.

Overall, the database configuration ensures reliable storage, efficient data retrieval, and smooth communication between the frontend and backend systems, making the application robust, scalable, and easy to maintain.

3.3.2 Schema Structure

The schema defines how data is structured in the database. In this project, two main schemas are used: Event Schema and Booking Schema. The Event schema includes fields such as event name, department name, date, time, venue, ticket price, and available tickets. The Booking schema contains user details like name, email ID, department, number of tickets booked, total amount, and booking timestamp. These schemas are defined using Mongoose, which enforces data structure and validation rules. The schema design ensures consistency and efficient data handling within the application.

3.3.3 Tables created

Since MongoDB is a NoSQL database, it uses collections instead of traditional tables. The project primarily includes two collections: Events and Bookings. The Events collection stores all event-related details, including ticket availability, while the Bookings collection stores records of each ticket booking made by users. Each document in the Bookings collection is linked to an event, ensuring proper tracking of bookings. This structure allows efficient storage, retrieval, and updating of data, supporting real-time ticket management in the application.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a client–server architecture, where the application is divided into three main layers: frontend, backend, and database. The frontend is developed using React (JavaScript library), which handles the user interface and user interactions. It displays event details and provides forms for ticket booking. The frontend communicates with the backend by sending HTTP requests (API calls) and receiving responses in JSON format. The backend is built using Node.js and Express.js, which handle business logic, request processing, and validation. When a user submits a booking request, the backend checks the availability of tickets, validates the data, and processes the request accordingly. It then sends a response back to the frontend with booking confirmation or error messages. The database layer uses MongoDB, where all event details and booking records are stored. The backend interacts with the database to retrieve event data and update ticket availability after each booking. This ensures real-time synchronization of data across the system. Overall, the architecture ensures clear separation of concerns, scalability, and efficient data handling, making the application reliable and easy to maintain.

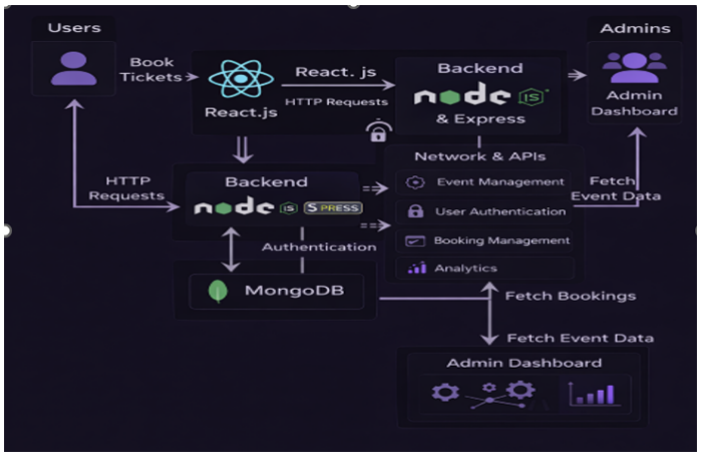

Figure 4.1: Project Architecture

[Figure 4.1] The diagram illustrates the architecture of a ticket booking system, where users interact with a React.js frontend that communicates with a Node.js and Express backend. The backend handles authentication, event management, and data storage in MongoDB, while admins access dashboards to monitor bookings and manage events.

4.2 DataFlow Diagram

The Data Flow Diagram (DFD) represents how data moves within the ticket booking system. The user interacts with the frontend interface to view event details and submit booking information. The frontend sends requests to the backend server through APIs. The backend processes the request, validates the input, and interacts with the database to fetch or update event and booking data. The database stores event details and booking records, and the backend returns the response to the frontend. Finally, the frontend displays the booking confirmation and updated ticket availability to the user. This flow ensures smooth communication between all components of the system.

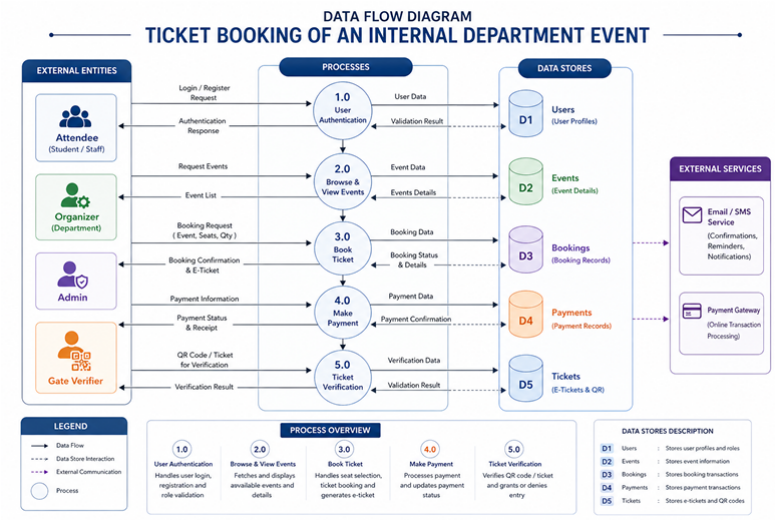

Figure 4.2: DataflowDiagram

[Figure 4.2] This figure illustrates the flow of data within the ticket booking system. It shows how user inputs are processed through different stages such as validation, booking, and confirmation. The diagram highlights interactions between users, system processes, and database. It ensures a clear understanding of how data moves across the system.

4.3 Module 1:Event Details Module

This module is responsible for displaying all event-related information to the user. It includes details such as event name, department name, date and time, venue, ticket price, and available tickets. The module fetches data from the backend and presents it in a user-friendly format. It helps users understand the event before booking tickets and ensures transparency in ticket availability.

4.4 Module 2:Ticket Booking Module

The Ticket Booking Module allows users to book event tickets by entering details like name, email ID, department, and number of tickets. It provides a simple and user-friendly interface for easy booking. The system validates all inputs to ensure

correct and complete information. It checks email format and prevents invalid or empty entries. The module ensures users cannot book more tickets than available. After validation, the request is sent to the backend for processing. The backend verifies availability and confirms the booking. Users receive confirmation and error messages based on the booking status.

4.5 Module 3:Booking Confirmation Module

The Booking Confirmation Module displays a confirmation message after a successful ticket booking. It provides a clear summary of the booking details to the user. The summary includes user name, event name, and number of tickets booked. It also shows the total amount paid for the booking[5]. The module updates and displays the remaining available tickets in real-time. It ensures transparency by confirming that the booking is successfully completed. Users receive clear feedback about their booking status. This module improves user experience by providing accurate and immediate confirmation.

Advantages of this Project

Content

a) TUser-Friendly Interface: The application provides a simple and easy-to-use interface for booking tickets.

b) Real-Time Updates: Ticket availability is updated dynamically after each booking.

c) Efficient Validation: Ensures accurate user input and prevents overbooking.

Disadvantages

The project is designed for small-scale use and may not support large-scale event management without further enhancements.

Chapter 5

RESULTS AND DISCUSSIONS

The developed React JS–based ticket booking application successfully achieves its objective of providing an efficient and user-friendly platform for internal event ticket booking. The system allows users to view event details and book tickets seamlessly, with proper validation ensuring accurate input and preventing overbooking. The use of dynamic state management enables real-time updates of available tickets, which enhances the reliability of the system.

During testing, the application performed well in handling user inputs, displaying appropriate error messages for invalid entries, and generating correct booking confirmations with summary details. The modular component structure improved code organization and made the application easy to maintain and extend. Integration between frontend, backend, and database ensured smooth data flow and consistent updates.

The results indicate that the system is suitable for small-scale event management within academic environments. However, for larger-scale deployment, additional features such as user authentication, payment integration, and enhanced security mechanisms can be incorporated. Overall, the project demonstrates the effective use of React and modern web technologies in building a responsive and interactive web application.

Table 5.1: Comparison of Ticket Booking Applications

Applications	**Features**
BookMyShow	Online booking, seat selection, payment integration
Eventbrite	Event management, ticket sales, analytics
Townscript	Workshop/event booking, simple UI
Paytm Insider	Event discovery, booking, notifications
Proposed System	Simple booking, validation, real-time updates

[Table 5.1] The table presents a comparison of different ticket booking applications and their key features. It highlights functionalities such as online booking, event management, seat selection, and notification services across platforms. This comparison helps in understanding the strengths of existing systems and the improvements offered by the proposed system.

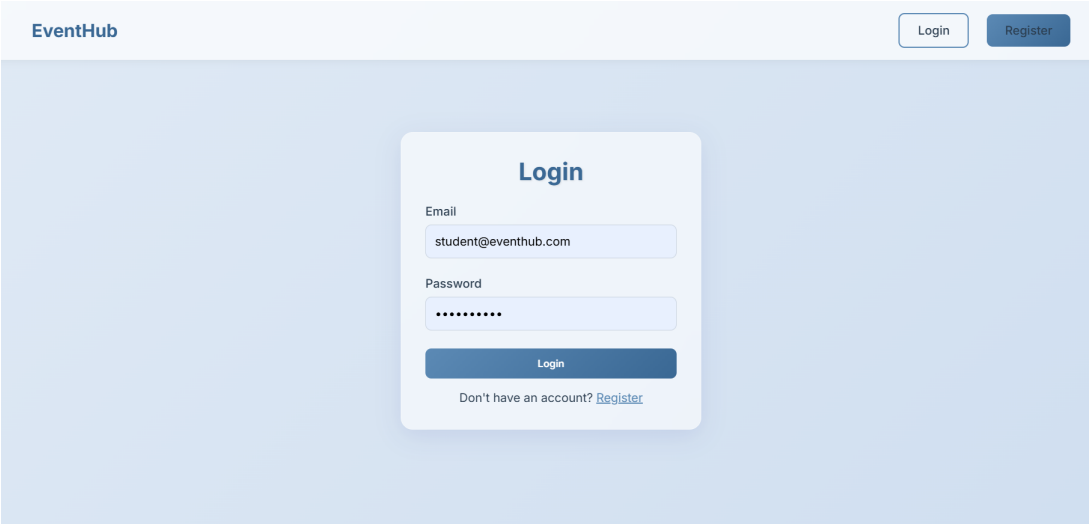

Figure 5.1: Login page

[Figure 5.1] This figure shows the login interface of the application. It allows users to enter their email and password to access the system. The design is simple and user-friendly, ensuring easy navigation. It also provides an option for new users to register.

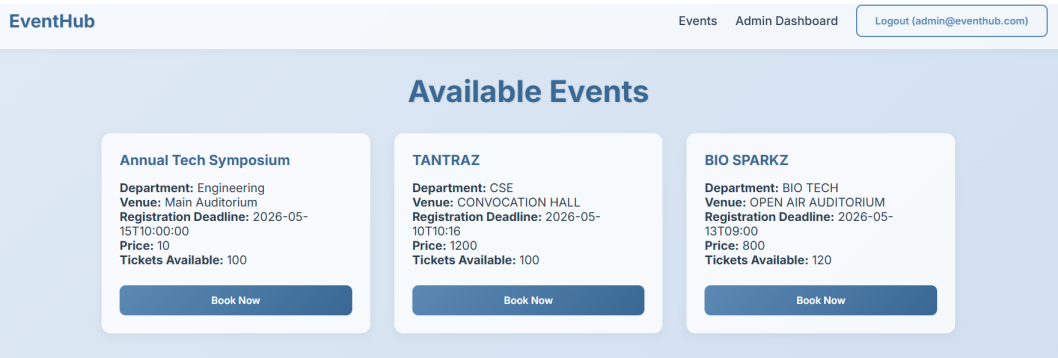

Figure 5.2: Dashboard page

[Figure 5.2] This figure displays the dashboard where available events are listed. It shows event details such as name, department, date, and ticket availability. Users can easily select and book tickets from this page. The interface provides a clear overview of all events.

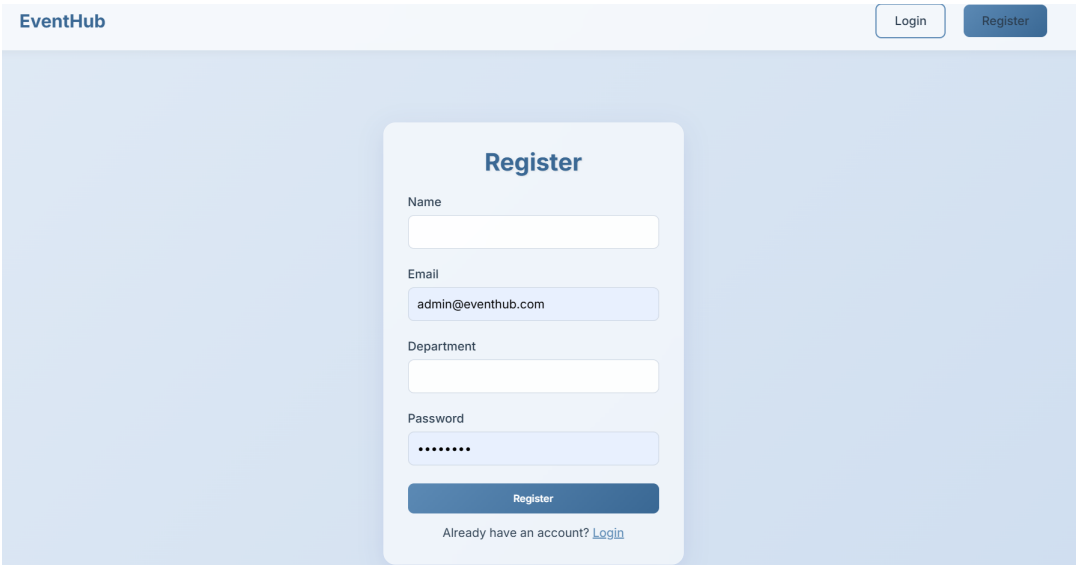

Figure 5.3: Registration Page

[Figure 5.3] This figure represents the user registration interface. It allows new users to create an account by entering details like name, email, and password. The form ensures proper input validation for accuracy. It provides a simple and secure way to register in the system.

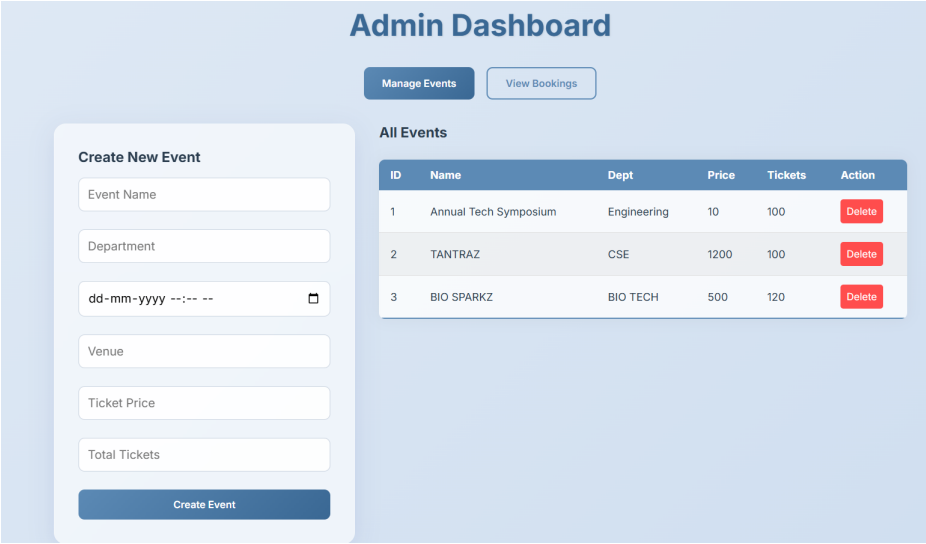

Figure 5.4: Admin Dashboard

[Figure 5.4] This figure shows the Admin Dashboard of the ticket booking system. It provides an interface for administrators to manage events efficiently. The dashboard includes options such as “Manage Events” and “View Bookings.” On the left side, a form is provided to create new events. The form collects details like event name, department, date, venue, price, and total tickets. On the right side, a table displays all existing events. Each event entry includes information such as ID, name, department, price, and available tickets. Administrators can delete events using the action button provided. The layout is clean and user-friendly, making it easy to manage event data. This dashboard improves administrative control and simplifies event management operations.

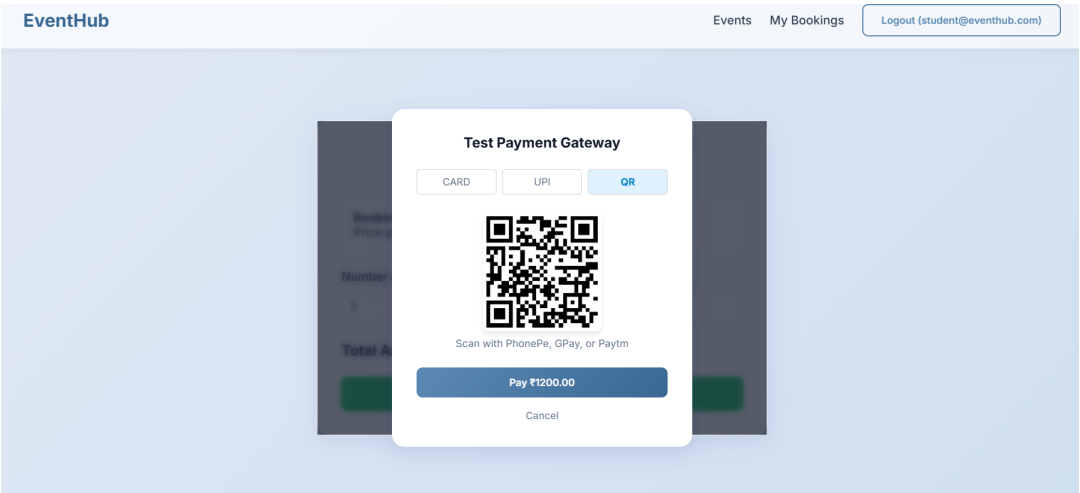

Figure 5.5: Payment page

[Figure 5.5] This figure shows the payment interface of the ticket booking system. It provides users with multiple payment options such as card, UPI, and QR code. The interface displays a QR code that can be scanned using mobile payment applications. Users can choose their preferred payment method for convenience. The design is simple, clean, and user-friendly for easy navigation. It ensures a smooth and secure payment process. The system displays the total amount to be paid before confirmation. After successful payment, the booking process is completed. A confirmation message is generated to inform the user about the transaction status. This module enhances the overall user experience by providing reliable and fast payment functionality.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The React JS–based ticket booking application provides a simple and efficient platform for managing internal department events. It enables users to easily browse event details and complete ticket bookings without complexity. The system ensures real-time updates of ticket availability, improving accuracy and reliability. Instant booking confirmation enhances the overall user experience and reduces uncertainty. Proper validation mechanisms are implemented to prevent errors and avoid over-booking issues. The user interface is clean, responsive, and easy to navigate for all types of users. Modern web technologies and reusable components improve performance and scalability. The integration between frontend and backend is smooth and efficient. The system is designed to be flexible for future enhancements and feature additions. Overall, the application successfully meets its objectives and serves as a dependable event management solution.

6.2 Future Enhancements

Future enhancements can significantly improve the functionality and scalability of the system. User authentication and secure login features can be added to provide personalized and protected access for users. Integrating online payment systems will enable safe and convenient transactions. The system can also be extended to support multiple events across different departments or organizations, making it more versatile. Additionally, email and SMS notifications can be implemented to keep users informed about booking confirmations, cancellations, and updates. To further enhance security and management, QR code–based ticket generation and verification can be introduced for easy and reliable entry control. An admin dashboard can be developed to efficiently manage events, monitor bookings, and control system operations. Role-based access can also be implemented to differentiate permissions between administrators and users. Real-time analytics will help track user activity, booking trends, and system performance, providing valuable insights. Moreover, integrating cloud-based databases can improve data storage, backup, and reliability. Deploying the application on live servers will make it accessible to a wider audience. Enhancing mobile responsiveness and refining the UI/UX design will ensure a smooth and engaging user experience across devices. Overall, these enhancements will make the system more robust, scalable, and suitable for real-world applications. Additionally, integration with third-party APIs can expand the system's capabilities, such as location services and calendar synchronization. Support for multilingual interfaces can make the application accessible to a broader range of users. Automated backup and recovery mechanisms can be implemented to ensure data safety and system reliability. Incorporating AI-based recommendations can help users discover relevant events based on their interests. Regular system updates and maintenance will further enhance performance, security, and long-term sustainability.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Online Ticket Booking System to Complex Engineering Problem (CEP)

Level	**Attribute**	**Characteristics**	**WKs**	**Justification**
WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	Knowledge of React JS, hooks, validation, UI design
WP2	Conflicting Requirements	Technical & non-technical constraints	WK4, WK5	Balances usability with booking ticket constraints and validation
WP3	Depth of Analysis	Analytical & design thinking	WK3, WK4	Component design, state handling, dynamic updates
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Students, faculty, organizers
WP7	Interdependence	System-level integration	WK3, WK5	UI + state + validation integration

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Online Ticket Booking System to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking
EA5	Familiarity	Beyond standard	Dynamic real-time system

7.3 SDG Mapping

Table 7.3: SDG Mapping for the Online Ticket Booking System

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Supports academic events
SDG 9	Innovation	Promotes React-based systems
SDG 10	Reduced Inequalities	Accessible to all users
SDG 11	Sustainable Communities	Encourages organized events

REFERENCES

1. Meta Platforms Inc. (2024). *React: A JavaScript Library for Building User Interfaces*. Available at: https://react.dev/https://react.dev/

2. OpenJS Foundation (2024). *Node.js Runtime Environment Documentation*. Available at: https://nodejs.org/en/docshttps://nodejs.org/en/docs

3. Oracle Corporation (2024). *MySQL Relational Database Management System*. Available at: https://dev.mysql.com/doc/https://dev.mysql.com/doc/

4. Spring Boot Documentation (2024). *Spring Boot Framework for Java Applications*. Available at: https://docs.spring.io/spring-boot/index.htmlhttps://docs.spring.io/spri boot/index.html

5. Razorpay (2024). *Payment Gateway Integration Documentation*. Available at: https://razorpay.com/docs/https://razorpay.com/docs/

6. Bootstrap Team (2024). *Bootstrap Frontend Framework Documentation*. Available at: https://getbootstrap.com/docs/https://getbootstrap.com/docs/

7. MDN Web Docs (2024). *HTML, CSS, and JavaScript Documentation*. Available at: https://developer.mozilla.org/https://developer.mozilla.org/

8. BookMyShow (2024). *Online Ticket Booking Platform*. Available at: https://in.bookmysho Official Website